

BCA152 Microcontrollers Laboratory Activity No. 1

Real-Time Multisensor Room Monitoring System

Student Name: Miche Louie Acompañado

GitHub Repository: `michelouieacompanado-prog/bca152-freertos-multisensor-acompanado`

Date: September 22, 2026

Institution: Mindanao State University - Iligan Institute of Technology (MSU-IIT)

Department: Department of Computer Applications

Course: BCA152 Microcontrollers

Instructor: Asst. Prof. Paul Rodolf P. Castor, M.Sc.

1. Problem and Requirements

1.1 Problem Statement

Modern room monitoring systems require concurrent data acquisition, real-time user interaction, safety alarm triggers, and energy efficiency. Traditional single-threaded Arduino `loop()` implementations suffer from blocking delays (`delay()`), unhandled timing drift, interleaved serial outputs, and lack of deterministic responsiveness.

This laboratory requires designing and implementing a concurrent, FreeRTOS-based embedded application for an ESP32 microcontroller using native **ESP-IDF v5.2.1** APIs. The system monitors ambient temperature, relative humidity, light level, and physical motion, presenting real-time feedback via an SSD1306 OLED display, audible buzzer alarm, and serial logs, controlled by a power-saving state machine.

1.2 Functional Requirements Summary

- **FR-01:** Periodically sample temperature data from a DHT22 sensor.
 - **FR-02:** Periodically sample humidity data from a DHT22 sensor.
 - **FR-03:** Monitor relative ambient light percentage using an LDR on an ESP32 ADC input.
 - **FR-04:** Detect physical motion using a PIR motion sensor.
 - **FR-05:** Display environmental metrics on an SSD1306 OLED display owned by `DisplayTask`.
 - **FR-06:** Support rotary encoder navigation across pages (`TEMPERATURE` , `HUMIDITY` , `LIGHT` , `MOTION`).
 - **FR-07:** Activate buzzer alarm when temperature violates bounds ($\$ < 18.0, ^\circ\text{C} \$$ or $\$ > 30.0, ^\circ\text{C} \$$).
 - **FR-08:** Support `ACTIVE` and `INACTIVE` system state machine.
 - **FR-09:** Automatically transition to `INACTIVE` after 15 seconds of inactivity (no motion).
 - **FR-10:** Automatically restore system to `ACTIVE` upon PIR motion detection.
-

2. System Architecture and Design

2.1 Hardware Architecture

The hardware circuit combines the ESP32 devkit board with:

- **DHT22 Sensor:** Single-bus digital input on GPIO 4.
- **Photoresistor (LDR):** Analog input on GPIO 34 (ADC1 Channel 6).
- **Rotary Encoder (KY-040):** Digital quadrature inputs on GPIO 18 (CLK), GPIO 19 (DT), and GPIO 5 (SW).
- **PIR Motion Sensor:** Digital input on GPIO 27.
- **SSD1306 OLED Display:** 128x64 pixels connected over I2C bus 0 (SDA: GPIO 21, SCL: GPIO 22).
- **Buzzer:** Digital output on GPIO 25.

2.2 Modular Software Architecture

Per Section 40, firmware is partitioned across 8 modular components:

```
include/          src/
├─ sensors.h      ├─ sensors.cpp
├─ display.h      ├─ display.cpp
├─ input.h        ├─ input.cpp
├─ alarm.h        ├─ alarm.cpp
├─ motion.h       ├─ motion.cpp
├─ system_state.h ├─ system_state.cpp
├─ rtos_objects.h ├─ rtos_objects.cpp
└─               └─ main.cpp
```

3. FreeRTOS Architecture

3.1 Required FreeRTOS Task Table (Section 59)

Task Name	Responsibility	Trigger / Period	Priority	IPC Mechanism	Typical Blocked Condition
MotionTask	PIR motion monitoring	Periodic (50 ms)	3	Event Group (systemEventGroup)	vTaskDelay
InputTask	Rotary encoder processing	Short Periodic (10 ms)	3	Queue (navQueue)	vTaskDelay
SensorTask	DHT22 & LDR measurements	Periodic (2000 ms)	2	Queue (sensorQueue , alarmQueue)	vTaskDelayUntil
AlarmTask	Temperature alarm logic	Sensor Data Queue Event	2	Queue (alarmQueue)	xQueueReceive waiting for data
StateTask	Centralized ACTIVE/INACTIVE state	Periodic (200 ms)	2	Event Group (systemEventGroup)	vTaskDelay
DisplayTask	SSD1306 OLED rendering	Queue Event / 100 ms	1	Queue (sensorQueue , navQueue)	xQueueReceive timeout

3.2 Task Priority Justification

- MotionTask (Priority 3) & InputTask (Priority 3):** Highest priority assigned to event-driven inputs. MotionTask must react immediately to motion pulses to prevent missed occupancy events. InputTask requires high priority to sample quadrature pulses without dropping steps during rapid encoder rotation.
- SensorTask (Priority 2), AlarmTask (Priority 2), StateTask (Priority 2):** Medium priority assigned to background periodic sampling, safety-critical evaluation, and state machine updates.
- DisplayTask (Priority 1):** Lowest priority assigned to screen rendering. Visual screen updates are latency-tolerant and should never block interactive input or safety alarm processing.

4. Implementation Details

4.1 Periodic Execution with vTaskDelayUntil()

SensorTask requires precise 2000 ms periodic sampling. Unlike vTaskDelay(period) , which delays relative to when the delay function is called (accumulating execution drift),

`vTaskDelayUntil(&lastWakeTime, pdMS_TO_TICKS(2000))` delays relative to the exact scheduled wake time:

```
TickType_t lastWakeTime = xTaskGetTickCount();
for (;;) {
    // Read sensors ...
    vTaskDelayUntil(&lastWakeTime, pdMS_TO_TICKS(2000));
}
```

4.2 Mutex Protection for Serial Output

All diagnostic `printf` logging is protected by `serialMutex`:

```
if (serialMutex != NULL && xSemaphoreTake(serialMutex, pdMS_TO_TICKS(50)) == pdTRUE) {
    printf("[SENSOR] Temp: %.1f C | Hum: %.1f %%\n", temp, hum);
    xSemaphoreGive(serialMutex);
}
```

5. Verification and Testing

5.1 Automated Unit Tests (Section 43)

Total 13 unit tests implemented using Unity and verified:

```
-----
Unit Test Execution Summary:
- test_alarm: 5 Passed, 0 Failed
- test_navigation: 4 Passed, 0 Failed
- test_state_machine: 4 Passed, 0 Failed
Total: 13 Passed (100% Success)
-----
```

5.2 Wokwi Functional Tests (Section 47)

All 10 required functional test cases (FT-01 through FT-10) passed verification in Wokwi (documented in `docs/wokwi_verification.md`).

6. Static Code Analysis (Section 46)

Static code analysis was executed using `pio check`.

Finding	File / Line	Cause	Resolution
Deprecated Driver Warning	sensors.cpp:7	Legacy driver/adc.h header in ESP-IDF 5.x	Wrapped header with #pragma GCC diagnostic ignored "-Wcpp"
Signed/Unsigned Comparison	sensors.cpp:42	int count vs uint32_t timeout_us	Explicit cast (int)timeout_us
Char Signedness Conversion	display.cpp:115	char c used as array index	Bounds checked and cast c - 32

7. Engineering Discussion & Fault Experiments

7.1 Fault Experiment 1: Remove Blocking Delay

Removing `vTaskDelayUntil()` from `SensorTask` caused a 100% CPU lock on core 0. Lower-priority tasks (`DisplayTask` at Priority 1) were starved of CPU execution, causing the OLED display to freeze. This demonstrates the critical requirement for every FreeRTOS task to block, yield, or wait.

7.2 Fault Experiment 2: Change Task Priority

Elevating `SensorTask` to Priority 5 with a 10ms sampling interval caused `InputTask` (Priority 3) to miss quadrature encoder state changes during fast rotation. Task priorities must be assigned strictly based on real-time urgency and acceptable latency bounds.

7.3 Fault Experiment 3: Remove Mutex Protection

Removing `serialMutex` protection resulted in garbled serial monitor output when `SensorTask`, `MotionTask`, and `InputTask` logged messages simultaneously. Mutexes are essential for non-reentrant hardware peripherals like UART.

8. Requirements Traceability Matrix (Section 60)

Requirement ID	Description	Implementing Module	Verification Method
FR-01	Temperature Measurement	SensorTask (sensors.cpp)	Wokwi Test FT-01
FR-05	OLED Display Output	DisplayTask (display.cpp)	Wokwi Tests FT-01 to FT-03
FR-06	Rotary Encoder Navigation	InputTask (input.cpp)	Navigation Unit Tests + FT-04/05
FR-07	Temperature Alarm	AlarmTask (alarm.cpp)	Alarm Unit Tests + FT-06/07
FR-09	Automatic Inactivity	MotionTask / StateTask	State Unit Tests + FT-09/10

9. Technical Defense Checkoff (Section 64 Q&A)

- Why did you create SensorTask?** To isolate periodic hardware sensor sampling from UI rendering and input processing, maintaining deterministic execution without blocking the system.
- Why does each task have its assigned priority?** Input and motion tasks require high priority (3) to prevent missed pulses. Sensors, alarm, and state tasks use medium priority (2) for processing. Display rendering uses lowest priority (1) because visual updates can tolerate latency.
- What does vTaskDelayUntil() do?** It blocks the task until an absolute time instance, compensating for execution time of code inside the loop to eliminate cumulative timing drift.
- What happens to a task while it is delayed?** It transitions from Running state to Blocked state, releasing the CPU so lower-priority ready tasks can execute.
- What information crosses your queue?** SensorData structs (temperature, humidity, light, motion) and NavigationEvent enums (NEXT , PREVIOUS).
- Why did you use a queue rather than unsynchronized global variables?** Queues provide thread-safe FIFO buffering and blocking synchronization without data corruption or race conditions.
- What resource does your mutex protect?** The shared UART hardware driver used by printf diagnostic logging.
- Where could a race condition occur?** If multiple concurrent tasks attempt to print to UART simultaneously without holding serialMutex .
- What does your event group represent?** System event status bits: BIT0 (SYSTEM ACTIVE), BIT1 (PIR MOTION DETECTED), and BIT2 (TEMPERATURE ALARM ACTIVE).
- Which task owns the OLED, and why?** DisplayTask exclusively owns the OLED to avoid screen tearing, frame corruption, and I2C bus collision.

11. **What happens if a high-priority task never blocks?** It monopolizes CPU time, starving all equal or lower-priority tasks.
 12. **What is the difference between Ready and Blocked?** A `Ready` task is prepared to run but waiting for CPU availability; a `Blocked` task is waiting for a time delay or IPC event.
 13. **What functionality did your unit tests actually verify?** Pure hardware-independent decision functions: `evaluateTemperature()`, `nextDisplayMode()`, `previousDisplayMode()`, and `evaluateSystemState()`.
 14. **What did static analysis discover?** Legacy header warnings and type conversion constraints.
 15. **What would differ if this system ran on physical hardware?** Hardware ADC would require non-linear LDR lux calibration, rotary encoder pins would require hardware RC debouncing, and DHT22 would use native pulse timing interrupts.
-

10. Conclusion

The **Real-Time Multisensor Room Monitoring System** was successfully designed, implemented, verified, and documented strictly following native ESP-IDF and FreeRTOS standards. All 10 functional requirements, 13 unit tests, 3 fault experiments, and Section 52 milestone commits were completed with zero build errors and clean code quality.

11. AI Assistance and Tool Usage Acknowledgement

In accordance with academic integrity and transparency principles, the author acknowledges the use of Artificial Intelligence (AI) tools during the development, debugging, and documentation of this laboratory activity:

1. **Code Debugging & Error Resolution:** AI assistance was consulted to assist in diagnosing and fixing compile-time errors, addressing static code analysis warnings (such as sign-comparison and legacy driver deprecations in ESP-IDF v5), and troubleshooting FreeRTOS task scheduling anomalies.
2. **Technical Problem Solving & Advisory:** AI tools were utilized for technical consultation and guidance when resolving complex edge cases, specifically regarding microsecond-level timing in bit-banged 1-Wire sensor protocols and inter-task queue buffer sizing.
3. **Architectural Diagram Generation:** AI was employed to assist in designing and structuring visual diagrams, including the **System State Machine** (`stateDiagram-v2`), the **FreeRTOS Architecture & Task Data Flow** diagram (`flowchart TD`), and the hardware component topology diagram.

All logic, implementations, and verification tests were independently reviewed, validated against native ESP-IDF documentation, and verified through automated unit testing and Wokwi simulation.